

Contents

The Wordle Solver: Algorithms and Implementation	1
1. The problem, formalized	1
2. The objective function	2
3. Scoring: the base-3 two-pass algorithm	2
4. Precomputation: the score matrix and its transpose	3
5. Hashing: 128-bit Zobrist	4
6. Lower bounds	4
7. The branch-and-bound search	5
7.1 The <code>s2</code> trick (variance in one pass)	6
8. Transposition tables	6
9. The greedy aspiration heuristic	7
10. Move ordering and the candidate beam	8
11. Sorting: introsort and partial selection	8
12. Parallelism	8
13. Decision-tree JSON export	9
14. Memory auto-tuning	9
15. Putting it all together	9
16. Exactness caveats	9
17. Further reading	10
18. References	10

The Wordle Solver: Algorithms and Implementation

A deep, code-annotated tour of `src/wordle_solver.c` — an exact **easy-mode** Wordle solver that computes the mathematically optimal decision tree (minimum total guesses, equivalently minimum average score) over a 14,855-word guess list and a 3,209-word answer list.

This document explains *what* each algorithm does, *why* it is correct, and *how* the C code implements it. It assumes comfort with basic combinatorics, big-O reasoning, and game-tree search. Line references are into `src/wordle_solver.c` (3,307 lines).

1. The problem, formalized

There are two finite sets of five-letter words:

- G , the **guess list** (every word you are allowed to play), $|G| \approx 14,855$.
- $T \subseteq G$, the **answer list** (the words the hidden answer may be), $|T| \approx 3,209$.

A **score function** $\sigma(g, t) \in \{0, \dots, 242\}$ reports how close a guess g is to the answer t (242 = “exactly correct”). A **strategy** is a decision tree: at each node holding a candidate set $S \subseteq T$, it names a guess $g \in G$; the true answer splits S into up to 243 buckets

$$S_{g,s} = \{t \in S : \sigma(g, t) = s\},$$

and the tree recurses into the bucket the true answer fell into. When S is a singleton you are done.

“Easy mode” means the next guess may be *any* word in G — it is **not** constrained to remain consistent with the scores already received. That single assumption is what makes the whole search tractable, and the report returns to it several times.

Solver goal: find a decision tree minimizing

$$\sum_{t \in T} \text{guesses}(t),$$

i.e. the *total* number of guesses summed over all answers, which is the average score times $|T|$.
The CLI reports the average (`exact_avg_score`).

2. The objective function

Let $V(S)$ be the minimum total guesses needed to identify every word in a candidate set S . It obeys the recurrence

$$V(S) = \min_{g \in G} \left(|S| + \sum_{s \neq 242} V(S_{g,s}) \right), \quad V(\emptyset) = 0, \quad V(\{t\}) = 1. \quad (1)$$

Why $|S|$? Each target in S consumes the current guess (one unit each). The 242 bucket is excluded from the sum because it holds only $t = g$ itself — that answer is already identified by the current guess, so it contributes nothing further. Every other non-empty bucket recurses.

Two immediate consequences.

1. **Separable at the root.** The value of a *fixed* first guess g is $|T| + \sum_{s \neq 242} V(T_{g,s})$. The search can therefore (a) rank guesses by a cheap estimate, (b) compute the root partition once, and (c) solve each bucket independently — which is exactly how the parallel mode (§12) decomposes the work.
2. **Superadditivity $\rightarrow$ valid additive lower bounds.** For **disjoint** sets A, B , $V(A \cup B) \geq V(A) + V(B)$. Proof sketch: take an optimal tree for $A \cup B$; the path it plays against any answer $a \in A$ is a valid strategy for solving A alone, so $V(A) \leq$ (that tree’s total cost restricted to A), and likewise for B . Hence $V(A) + V(B) \leq V(A \cup B)$. This justifies summing lower bounds of disjoint buckets (§6, §7.7) and is why the recursion never has to *prove* a combined bound from scratch.

Note this objective is *not* adversarial minimax. The feedback is a deterministic function of the answer; “minimax” here reduces to “minimize the total over all answers”, i.e. the expected number of guesses under a uniform prior. The pruning machinery (§7) is the analogue of alpha-beta, applied to this additive objective.

3. Scoring: the base-3 two-pass algorithm

`compute_score` (`src/wordle_solver.c:164-192`) is the 2-pass algorithm that reproduces Wordle’s (and Wordlebot’s) exact semantics, including the duplicate-letter rule.

```
static inline uint8_t
compute_score(const char *restrict guess, const char *restrict target)
{
    uint8_t counts[26] = {0};
    bool is_green[WORD_LEN];
    uint8_t score = 0;
    int i;

    /* Pass 1: greens; count only non-green target letters */
    for (i = 0; i < WORD_LEN; i++) {
        if (guess[i] == target[i]) {
            is_green[i] = true;
        } else {
```

```

        is_green[i] = false;
        counts[target[i] - 'a']++;
    }
}

/* Pass 2: left to right; greedy yellow assignment */
for (i = 0; i < WORD_LEN; i++) {
    if (is_green[i]) {
        score = (uint8_t)(score * 3 + 2);          /* green = 2 */
    } else if (counts[guess[i] - 'a'] > 0) {
        counts[guess[i] - 'a']--;                  /* consume a target letter */
        score = (uint8_t)(score * 3 + 1);          /* yellow = 1 */
    } else {
        score = (uint8_t)(score * 3 + 0);          /* gray = 0 */
    }
}
return score;
}

```

Each position i contributes a trit $s_i \in \{0, 1, 2\}$; the result is the base-3 integer

$$\sigma = s_0 \cdot 81 + s_1 \cdot 27 + s_2 \cdot 9 + s_3 \cdot 3 + s_4 = \sum_{i=0}^4 s_i 3^{4-i} \in \{0, \dots, 242\}.$$

The subtlety is duplicates. Pass 1 marks greens and counts the *unmatched* target letters (excluding greens). Pass 2 scans left-to-right and awards a yellow the first time a letter still has an unmatched occurrence, decrementing the counter — so `guess = "sheer"`, `target = "three"` scores 2 0 0 1 2 (one yellow, not two), matching the official game. This is a textbook **greedy left-to-right matching**; see §17 for the information-theoretic view.

The `EXACT_MATCH = 242` constant (all five greens) recurs throughout the code as the special “this guess *is* the answer” bucket.

4. Precomputation: the score matrix and its transpose

The search needs $\sigma(g, t)$ billions of times, so it is precomputed once into a `uint8_t` matrix, then stored **twice** — row-major and transposed — because the two hot paths read it in opposite orders (`src/wordle_solver.c:319-361`).

- `score_matrix[g * T + t]` — row-major, indexed by **guess**: used to *partition* a candidate set S into buckets (iterate $t \in S$ against a fixed g).
- `score_matrix_transposed[t * G + g]` — column-major, indexed by **target**: used in the candidate-ranking loop (iterate g while gathering each target’s score), so each target’s column is traversed sequentially, cache-line friendly.

Both are built with one pthread per row/column range. $14,855 \times 3,209 \approx 47.7\text{M}$ cells, two copies = ≈ 95 MB.

The Python side (`src/fast_scoring.py`) compiles its *own* score kernel for the online interactive solver; this C program is the offline exact-search engine and carries its own copy of the routine. They agree (see `src/test_solver.py`).

5. Hashing: 128-bit Zobrist

To memoize the value of a candidate set we need a collision-resistant **set hash**. The code uses **Zobrist hashing** with 128 bits: each answer word t is assigned a random 128-bit key $(z_1[t], z_2[t])$, and the hash of a set is the XOR of its members’ keys

$$H(S) = \left(\bigoplus_{t \in S} z_1[t], \bigoplus_{t \in S} z_2[t] \right).$$

XOR is the magic: it is order-independent and lets the hash of a *bucket* be maintained incrementally while histogramming a guess’s partition (§7.7), and lets two disjoint buckets be merged with a single XOR (§7.7). The keys are produced by **SplitMix64** (`src/wordle_solver.c:194-201`):

```
static uint64_t
splitmix64(uint64_t *state)
{
    uint64_t z = (*state += 0x9e3779b97f4a7c15ULL); /* golden-ratio increment */
    z = (z ^ (z >> 30)) * 0xbf58476d1ce4e5b9ULL;
    z = (z ^ (z >> 27)) * 0x94d049bb133111ebULL;
    return z ^ (z >> 31);
}
```

Two independent 64-bit streams (z_1 from one seed, z_2 from another) give a 128-bit key. The “128-bit” is deliberate: a single 64-bit key over the $\binom{3209}{n}$ possible candidate sets would invite birthday collisions at the tens-of-millions-of-nodes scale; 128 bits makes an accidental collision negligible for any realistic run (the file’s header calls this “zero collision risk”). Two *independent* hashes also let the transposition table compare h_1 and h_2 plus the set size, so even a deliberate h_1 collision is caught by h_2 .

6. Lower bounds

An **admissible** lower bound never exceeds the true value; the solver uses them to prune. `compute_lower_bound_table` (`src/wordle_solver.c:363-385`) precomputes a per-size bound $lb(k)$ for every $k \leq |T|$:

```
game->lower_bound[0] = 0;
for (k = 1; k <= n; k++) {
    if (k == 1)
        game->lower_bound[k] = 1;
    else if (k <= 243)
        game->lower_bound[k] = 2 * k - 1;
    else
        game->lower_bound[k] = 1 + 2*242 + 3*(k - 243);
}
```

Derivation (a “disjoint-union” bound):

- $k = 1$: you must name the word: 1 guess.
- $2 \leq k \leq 243$: one guess has at most 243 outcomes. Baseline k (one guess per target). At most one target can land in the free 242-bucket; the other $k - 1$ targets each need at least one more guess. Hence $lb(k) = k + (k - 1) = 2k - 1$.
- $k > 243$: beyond one 242-bucket there are only 242 other scores, so at most 242 targets can sit in singleton buckets (one further guess each); the remaining $k - 243$ targets must share buckets of size ≥ 2 , needing ≥ 2 further guesses. Hence $lb(k) = k + 242 + 2(k - 243) = 3k - 244$, written in the code as $1 + 2 \cdot 242 + 3(k - 243)$.

This bound is weak but *free to compute*, and it is combined with much sharper bounds found during the search:

- **Per-guess bound.** For a guess g partitioning S into buckets of sizes $m_1, \dots$,

$$LB(g) = |S| + \sum_{s \neq 242} lb(|S_{g,s}|).$$

Because $V(S_{g,s}) \geq lb(|S_{g,s}|)$ and V is additive over the disjoint buckets (§2), $LB(g) \leq$ true cost of g . The code computes this on the fly as `guess_lb`.

- **Memoized bounds.** Every visited node stores either its exact value or the best proven lower bound found so far in the transposition table (§8), tightening `lb` for sets that recur.

7. The branch-and-bound search

`solve_subset` (`src/wordle_solver.c:1174-1775`) computes $V(S)$ for a candidate set S given as a list of target indices, plus its 128-bit hash. It is a **fail-soft** branch-and-bound: the caller passes an aspiration bound β (an “upper bound we are trying to beat”); the function returns $V(S)$ if $V(S) < \beta$, otherwise it returns *some* lower bound $\geq \beta$ (which is still a correct, if not tight, lower bound).

```
solve_subset(S, beta):
  1. base cases (|S| = 0, 1, 2)
  2. transposition-table probe (exact? lower bound? suggested guess?)
  3. if node_lb >= beta: return node_lb           # fail-soft cutoff
  4. target-only instant resolution (|S| <= 243) # cheap "disjoint" checks
  5. rank all guesses; compute per-guess lb1      # move ordering
  6. if min_lb1 >= beta: return (fail-soft)
  7. if an exact upper bound == min_lb1: return it
  8. keep only guesses with lb1 < beta; partial-sort top max_candidates
  9. for each surviving guess (in rank order):
      partition S into buckets (histogram + XOR hashes)
      perfect-split / non-candidate shortcuts
      if guess_lb >= best: skip
      tier-2: resolve buckets from TT (deferred, no partition)
      tier-3: recurse on unresolved buckets with per-bucket beta
  10. return best (exact) or beta (fail-soft)
```

The body of the loop that makes the whole program correct is the **tier-3 recursion** (`src/wordle_solver.c:1695-1725`):

```
running_cost = resolved_cost;
remaining_lb = 0;
for (u = 0; u < num_unresolved; u++)
    remaining_lb += game->lower_bound[unresolved_buckets[u].size];

for (u = 0; u < num_unresolved; u++) {
    uint32_t sz = unresolved_buckets[u].size;
    remaining_lb -= game->lower_bound[sz];
    if (running_cost + remaining_lb >= current_best) { pruned = true; break; }

    bucket_beta = current_best - running_cost - remaining_lb;
    bucket_cost = solve_subset(solver, &local_partition[offset], sz,
                              h1, h2, bucket_beta, NULL, depth + 1);
    if (bucket_cost >= bucket_beta) { pruned = true; break; }
    running_cost += bucket_cost;
}
```

This is the heart of the additive bound. Buckets are processed in ascending size order, maintaining `running_cost` (sum of resolved buckets) and `remaining_lb` (sum of the *unprocessed* buckets’ static lower

bounds). Each bucket is handed a **fail-soft beta**

$$\beta_b = \text{current_best} - \text{running_cost} - \text{remaining_lb},$$

i.e. “to beat the current best, this bucket must come in strictly under the slack that remains”. If a bucket returns $\geq \beta_b$, the guess as a whole cannot improve **current_best**, so the entire guess is pruned.

A few other pruning devices inside the loop:

- **Perfect split shortcut** (:1574-1583). If the guess splits S into $|S|$ distinct buckets and one of them is the 242-bucket, then by §2 the cost is exactly $2|S| - 1$, which is optimal — record it and stop scanning.
- **Non-candidate singleton split** (:1586-1598). Same, but no 242-bucket: cost exactly $2|S|$.
- **Tier-2 deferred TT probe** (:1608-1674). Before materializing the partition array and sorting, each bucket is looked up in the transposition table using the hashes accumulated *during the histogram pass*. If every bucket is already exactly resolved, the guess’s exact cost is known immediately; if the summed lower bound already reaches **current_best**, the guess is pruned without allocating a partition at all. This avoids the $O(|S|)$ partition + **qsort** for most dead guesses.

Merged lower bounds (:1727-1750). When a guess is pruned *after* some buckets were exactly solved, those exact costs are not wasted: for every pair of solved disjoint buckets B_i, B_j , the code stores

$$lb(B_i \cup B_j) \geq cost(B_i) + cost(B_j), \quad H(B_i \cup B_j) = H(B_i) \oplus H(B_j),$$

exploiting superadditivity (§2) and XOR-mergeable hashes. Future nodes that encounter the union get a tighter lower bound for free.

7.1 The s2 trick (variance in one pass)

While ranking a guess, the code needs $\sum_b m_b^2$ (sum of squared bucket sizes), the “balance” statistic used for ordering (§10). It computes it **without** storing the histogram permanently, using the identity

$$\sum_b m_b^2 = \sum_{c=1}^m (2c - 1) \text{ summed over each bucket,}$$

i.e. as each target lands in a bucket whose running count goes $1, 2, \dots, m$, add $1, 3, 5, \dots$ (the odd numbers). In the unrolled fast path for $|S| \leq 8$ this is a single fused increment per target (**src/wordle_solver.c:1344**):

```
uint8_t sc0 = col0[g]; uint8_t c0 = ++counts[sc0];
s2 += 2 * c0 - 1;           /* 1 + 3 + 5 + ... = m^2 */
guess_lb += 2 - (c0 == 1);  /* +1 for 1st, +2 otherwise => lb(m) */
```

8. Transposition tables

A **transposition table** (TT) is a hash table memoizing (**candidate set** → **value**) so a set reachable by different guess orders is solved only once. The code uses a **two-tier** design (thread-local + lock-free shared) keyed by the 128-bit Zobrist hash and the size.

Local table — open addressing with linear probing, ≤ 16 probes (**src/wordle_solver.c:621-674**):

```
static inline TTEntry *
tt_find(TT *tt, uint64_t h1, uint64_t h2, uint32_t size)
{
    uint64_t idx = (h1 ^ h2) & tt->mask;
    for (probes = 0; probes < TT_MAX_PROBES; probes++) {
```

```

    TTEntry *e = &tt->entries[(idx + probes) & tt->mask];
    if (e->hash1 == TT_EMPTY_HASH && e->hash2 == TT_EMPTY_HASH) return NULL;
    if (e->hash1 == h1 && e->hash2 == h2 && e->size == size) return e;
}
return NULL;
}

```

Each entry stores three things: `exact_cost` (if the node was fully solved), a `proven_lower_bound`, and `best_guess` (used to order moves next time — §10).

The `shared` table is the interesting part: it is written by many threads concurrently with C11 atomics and **no locks** (`src/wordle_solver.c:744-793`). The trick is a seqlock-style handshake: `hash1` is the sentinel. A writer reserves an empty slot by CAS-ing `hash1` from `EMPTY` to `h1`, then fills `hash2`, `exact_cost`, `proven_lower_bound`, `best_guess`, and publishes `size` **last** with release semantics:

```

if (atomic_compare_exchange_strong(&e->hash1, &expected, h1, acq_rel, acquire)) {
    atomic_store(&e->hash2, h2, release);
    atomic_store(&e->exact_cost, exact_cost, release);
    atomic_store(&e->proven_lower_bound, proven_lb, release);
    atomic_store(&e->best_guess, best_guess, release);
    atomic_store(&e->size, size, release); /* published last */
    return;
}

```

A reader (`shared_tt_find`, :706-742) loads `hash1` with `acquire`; if it sees a full (`h1,h2,size`) match it may read the rest, guaranteed consistent because `size` was published last. A subtle consequence: a reader may briefly observe a half-written entry or miss a just-written one, but it can never observe a *torn* value. Since a TT is purely a cache of derived values (never a correctness requirement), this “eventually visible” semantics is perfectly fine and eliminates all locking overhead.

A local-TT hit does not re-fill the shared table; a shared hit is *promoted* into the local table (`solver_tt_find`, :963-985). Writes go to both.

9. The greedy aspiration heuristic

Branch-and-bound is only fast if it starts with a **tight upper bound**. The code seeds one with a one-ply greedy heuristic (`solve_greedy_tree`, `src/wordle_solver.c:1049-1130`): at every node, choose the guess that **minimizes the sum of squared bucket sizes**,

$$g^* = \arg \min_g \sum_b m_b^2(g),$$

then recurse into each bucket. Minimizing $\sum m_b^2$ maximizes how *balanced* the partition is, which is a fast proxy for information gain (see §10). The result is a complete — but not necessarily optimal — decision tree, giving a valid upper bound $UB(g)$ for any opener g . That bound is used two ways:

- as the **initial aspiration ceiling** for the whole search (`--top/--all` mode), and
- to give each root bucket a tight per-bucket β in the parallel opener evaluator (`evaluate_opener_parallel`, :1890-2005).

Because the bound is *valid* (it corresponds to a real, if greedy, strategy), pruning against it is sound.

10. Move ordering and the candidate beam

Order matters enormously: try good guesses first and the bound tightens quickly, pruning the rest. Each guess gets a 64-bit **key** that encodes its rank (src/wordle_solver.c:1395-1396):

```
rank_score = 2 * s2 + count * guess_lb - (in_set ? 2 : 0);
candidate_keys[g] = ((uint64_t)rank_score << 32) |
                    ((uint64_t)(guess_lb & 0xFFFF) << 16) |
                    (uint64_t)g;
```

where $s2 = \Sigma m_b^2$ (partition balance), `guess_lb` the per-guess lower bound (§6), and `in_set` true iff the guess is itself a candidate answer (a small bonus — a guess that might *be* the answer is worth slightly more). Lower `rank_score` is better: it first prefers balanced partitions, then small lower bounds, then in-set guesses. The key packs (`rank`, `guess_lb`, `index`) so a single integer partial sort orders candidates, with the TT's `best_guess` from a previous visit explicitly hoisted to the front (:1497-1507).

The beam. After dropping guesses whose `guess_lb` is $\geq \beta$, only the top `max_candidates` (CLI default 100, set via `--candidates/-n`, or unlimited with `--exhaustive`) are actually explored (:1494-1495):

```
limit = solver->max_candidates < m ? solver->max_candidates : m;
sort64_asc_top(candidate_keys, m, limit);
```

This is the one place the default configuration is *not* provably exact: the ranking is a heuristic, so the true optimal guess could in principle rank below the cut. In practice the balance+l.b. ranking is extremely reliable, but §16 spells out exactly when the result is guaranteed exact.

11. Sorting: introsort and partial selection

Two sorts are inlined and specialized to `uint64_t` to avoid `qsort`'s function-pointer call overhead in the hot loop:

- **sort64_asc** (:799-860) — an **introsort**: quicksort with a median-of-three pivot and explicit tail-recursion elimination (it recurses on the *smaller* partition and loops on the larger, bounding stack depth at $O(\log n)$), falling back to insertion sort for $n \leq 16$.
- **sort64_asc_top** (:862-924) — a **partial selection** (quickselect-like) that sorts *only* the k smallest elements into the first k positions, in $O(n + k \log k)$, exactly what the beam of §10 needs without paying for a full sort.

12. Parallelism

Parallelism is applied at three independent levels, each a classic work-queue over pthreads with C11 atomics:

1. **Matrix construction** (`init_game_data`, :319-361): the $G \times T$ score matrix and its transpose are computed by partitioning rows/columns across threads.
2. **Root-bucket parallelism** (`bucket_worker`, :1852-1888): once an opener's first guess is fixed, its 243 buckets are *independent* subproblems (§2). Workers atomically fetch the next bucket index, each running its own `Solver` with its own **local** TT, while sharing the **lock-free shared TT** (§8) so work discovered by one thread is reusable by all.
3. **Opener-pool parallelism** (`opener_worker`, :2496-2585): for `--top N / --all`, the *openers themselves* are farmed out over a shared work queue, with a shared `global_best_cost` atomic that keeps tightening as better openers are found, so later openers are solved against an ever-stronger ceiling (`evaluate_opener_sequential`, :2007-2094).

Each worker thread gets a large 8 MB stack (`PTHREAD_STACK_SIZE`) because the recursive search can run deep.

13. Decision-tree JSON export

`build_solution_tree` (:2259–2360) reconstructs the optimal tree by replaying `solve_subset` and recording the winning guess at each node, then `write_node_json` (:2375–2412) emits it as nested JSON keyed by score (0..242). The format is consumed by the Python `DecisionTreeStrategy` in `src/strategies.py`, so the exact tree can be shipped and evaluated instantly in the interactive solver — the C engine does the once-per-language search; Python just walks the tree.

14. Memory auto-tuning

`init_game_data` (:514–566) derives a cache budget from the host: by default 1/16th of physical RAM (clamped to 256 MB–1 GB), minus the matrix, split 65/35 between the shared and per-thread local TTs, each sized to a power of two. `--max-memory <MB>` overrides it.

15. Putting it all together

A single `--opener salet` run threads this path:

1. `load_wordlist` parses `data/words.txt` into `targets` (answers) and `guesses`.
 2. `init_game_data` builds the score matrix + transpose, Zobrist keys, the lower-bound table, and the TTs, sized to the RAM budget.
 3. `compute_opener_greedy_upper_bound` prices the opener with the greedy policy $\rightarrow$ seed β .
 4. `evaluate_opener_parallel` partitions the root, gives each bucket a fail-soft β , and farms buckets to workers.
 5. Each worker's `solve_subset` runs the full branch-and-bound: TT hits, target-only shortcuts, ranked beam, tier-2 TT probes, tier-3 recursion, merged lower bounds.
 6. Bucket costs are summed; the exact total and average are reported, and optionally a full decision tree is dumped as JSON.
-

16. Exactness caveats

- **Easy mode only.** The state is fully described by the candidate *set* S ; that is what makes hashing, memoization, and the additive bounds valid. Hard-mode (where the next guess must respect prior scores) has a richer state space and is not what this solver does.
- **Candidate beam.** With the default `max_candidates = 100`, `--opener/--top/--all` explore only the top-100 ranked guesses per node, so the result is near-optimal rather than *proven* optimal. Pass `--exhaustive` to disable the beam. The library entry point `wordle_subset_solve` and the `--subset` CLI mode always set `max_candidates` to `UINT32_MAX`, so they are exhaustive regardless of defaults.
- **Aspiration “pruned” results.** In the parallel opener path, if a bucket fails to beat its aspiration window, the opener is reported as **pruned** rather than re-solved exhaustively. The lower-bound/TT pruning itself is always sound.

The correctness of the *search kernel* (that `solve_subset` returns the true value when the beam is off) is pinned by `src/reference_solver.py` — an intentionally naive, obviously-correct brute-force oracle — against tiny word lists in `src/test_solver.py`, run under ASan, UBSan, and TSan.

17. Further reading

The game-theoretic / algorithmic foundations

- *Artificial Intelligence: A Modern Approach*, Russell & Norvig — Ch. 5 (adversarial search: minimax, alpha-beta pruning) and Ch. 3 (search) are the canonical background for everything in §7.
- *Introduction to Algorithms* (CLRS) — dynamic programming, hashing, and sorting; the DP view of §2 is exactly the “optimal binary search tree” style of recurrence.
- *The Art of Computer Programming*, Vol. 3 (Sorting and Searching) — hashing and the insertion-sort fallback; Vol. 4A (Combinatorial Algorithms) — backtracking and lower-bound pruning. Knuth’s *The Art of Computer Programming*, Vol. 4B (Dancing Links) is the definitive treatment of exact-cover search, the same family as this exact tree search.
- *Algorithms* (Sedgewick & Wayne) — a gentler route to the same sorting/searching.

Game-search specifics

- The [Chess Programming Wiki](#) is the single best free resource for the exact techniques used here: [Zobrist Hashing](#), [Transposition Table](#), [Alpha-Beta](#), [Aspiration Windows](#), [Fail-Soft](#), and [Move Ordering](#).
- Knuth & Moore, *An Analysis of Alpha-Beta Pruning* (1975) — the classical analysis of why good move ordering makes pruning dramatically effective.

Information theory (why the greedy metric works)

- Shannon, *A Mathematical Theory of Communication* (1948); see [Entropy](#). A balanced partition maximizes expected information $\mathbb{E}[\log \frac{1}{p}]$; minimizing $\sum m_b^2$ (§9) minimizes the *collision probability* $\sum p_b^2$ — the order-2 [Rényi entropy](#), which is monotone with Shannon entropy and far cheaper to compute.
- 3Blue1Brown, [Solving Wordle using information theory](#) — an accessible, rigorous tour of the same ideas in the Wordle setting.

Wordle-specific

- *Wordle is NP-hard*, Lokshtanov & Subercaseaux (2022), [arXiv:2203.16713](#) — why “exact optimal Wordle” is hard in general and the heuristics matter.
- Anderson, *The Math of Wordle* ([DukeMath blog](#)) — a clean write-up of the objective and known optimal openers.

Concurrency

- Herlihy & Shavit, *The Art of Multiprocessor Programming* — the theory behind the lock-free shared table (§8), CAS, and memory ordering.
- Preshing, *An Introduction to Lock-Free Programming* — an excellent practical companion.
- C11 atomic model: [cppreference.com](#) reference.

PRNG / hashing

- Steele, Lea & Flood, *Fast Splittable Pseudorandom Number Generators* (2014) — the source of Split-Mix64 (§5).
- Zobrist, *A New Hashing Method with Application for Game Playing* (1970) — the original Zobrist hashing paper.

18. References

1. A. L. Zobrist (1970). *A New Hashing Method with Application for Game Playing*. Technical Report #88, University of Wisconsin.
2. D. E. Knuth & R. W. Moore (1975). *An Analysis of Alpha-Beta Pruning*. *Artificial Intelligence* 6(4):293–326.
3. C. E. Shannon (1948). *A Mathematical Theory of Communication*. *Bell System Technical Journal* 27:379–423, 623–656.

4. G. L. Steele Jr., D. Lea, C. H. Flood (2014). *Fast Splittable Pseudorandom Number Generators*. OOPSLA 2014.
5. D. Lokshtanov & B. Subercaseaux (2022). *Wordle is NP-hard*. arXiv:2203.16713.
6. S. Russell & P. Norvig (2020). *Artificial Intelligence: A Modern Approach*, 4th ed., Pearson.
7. T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein (2022). *Introduction to Algorithms*, 4th ed., MIT Press.
8. D. E. Knuth. *The Art of Computer Programming*, Vols. 3 & 4A, Addison-Wesley.
9. M. Herlihy & N. Shavit (2012). *The Art of Multiprocessor Programming*, rev. ed., Morgan Kaufmann.
10. R. Sedgewick & K. Wayne (2011). *Algorithms*, 4th ed., Addison-Wesley.